Assignment 4

Multi-Client Chat Server using TCP

Submitted by

Jitupan Mondal

Roll Number: **EEB24023**

Contents

1 Objective

The objective of this assignment is to develop a multi-client chat application using TCP sockets where multiple clients can communicate through a centralized server. The assignment covers concurrent server design using threading, client management, message broadcasting, connection logging, and performance evaluation under varying client loads.

2 Network Topology

The experiment uses Mininet with a single-switch topology containing four hosts.

Host	Role
h1 (10.0.0.1)	Chat Server
h2 (10.0.0.2)	Client A
h3 (10.0.0.3)	Client B
h4 (10.0.0.4)	Client C

The topology is created using:

```
sudo mn --topo single,4
```

All four hosts connect through a single virtual switch `s1`. Connectivity was verified using:

```
mininet> nodes
mininet> net
mininet> pingall
```

The `pingall` command confirmed **0% packet drop** across all 12 host pairs, verifying that the network was fully operational before the experiment.

2.1 System Details

```
$ uname -a
Linux jitupan-mondal-QEMU-Virtual-Machine 7.0.0-22-generic
#22-Ubuntu SMP PREEMPT_DYNAMIC Mon May 25 15:37:49 UTC 2026 aarch64 GNU
    /Linux

$ python3 --version
Python 3.14.4
```

3 Program Design

3.1 Architecture Overview

The application follows a **hub-and-spoke model** where the server acts as the central hub. Clients never communicate directly; all messages flow through the server, which broadcasts them to every connected client.

```
Client A TCP Server TCP Client A (echo)
                        TCP Client B
                        TCP Client C
```

3.2 Server Design (`server.py`)

The server is built around four design decisions:

1. **Threading model:** One thread per connected client using `threading.Thread`. Each thread runs `handle_client()` and blocks on `recv()` independently, so a slow or blocked client does not stall the server.
2. **Client registry:** A Python dictionary `clients = {socket: username}` maps each socket to its registered username. This allows fast username lookup when a message arrives because the socket is already known.
3. **Thread safety:** A `threading.Lock()` named `clients_lock` protects the dictionary. Without this, two threads modifying the dictionary simultaneously may cause `RuntimeError: dictionary changed size during iteration`.
4. **Broadcast safety:** The broadcast function iterates over `list(clients.keys())`, which creates a snapshot copy so removals during iteration do not crash the loop.

3.3 Client Design (`client.py`)

The client uses two threads:

- **Main thread:** Reads user input using `input()` and sends it to the server.
- **Receive thread:** Continuously calls `sock.recv()` and prints incoming messages. It runs as a daemon thread.

This two-thread design is necessary because both `input()` and `recv()` are blocking operations. Running them in the same thread would freeze either sending or receiving.

3.4 Key Socket Calls Explained

Call	Purpose
`socket.AF_INET, SOCK_STREAM`	IPv4 + TCP (reliable, connection-oriented)
`setsockopt(SO_REUSEADDR, 1)`	Allows quick server restart without "Address already in use" error
`bind((HOST, PORT))`	Reserves port 5000 on the server machine
`listen(5)`	Queues up to 5 pending connections
`accept()`	Blocks until a client connects and returns a new socket
`sendall()`	Sends the full buffer reliably
`recv(4096)`	Receives up to 4096 bytes; empty bytes means disconnect

4 Experimental Results

4.1 Task 1: Server Connection Logs

The server logs all connection and disconnection events to `server_log.txt` in the required format `timestamp,EVENT,username,client_ip`.

```
21:25:09,CONNECTED,jitupan client1,10.0.0.2
21:28:23,DISCONNECTED,jitupan client1,10.0.0.2
23:11:29,CONNECTED,Rahul,10.0.0.2
23:12:12,CONNECTED,Bishal,10.0.0.3
23:12:52,CONNECTED,Priya,10.0.0.4
```

4.2 Task 2: Chat Log

All user messages are stored in `chat_log.txt` in the required format `timestamp,username,message`.

```
23:13:12,Rahul,Hello Everyone
23:13:47,Bishal,TCP sockets are powerful!
```

4.3 Task 3: Performance Measurement

Experiments were performed with 1, 2, and 3 clients, where each client sent 20 messages. The delivery time was measured using `time.perf_counter()` from the instant of sending a message to the instant the client received its own broadcast echo back from the server.

Clients	Total Messages	Avg Delivery Time (ms)	Throughput (msgs/sec)
1	20	0.485	2063.558
2	40	0.528	1892.506
3	60	0.606	1651.028

Observation: The average delivery time increases slightly as the number of clients increases, while throughput decreases moderately. This is expected because each incoming message must be broadcast to more recipients, increasing the total work performed by the server for each message.

5 Wireshark Verification

Traffic was captured using Wireshark with the display filter `tcp.port == 5000` on the Mininet interface used for the chat experiment.

5.1 TCP Three-Way Handshake

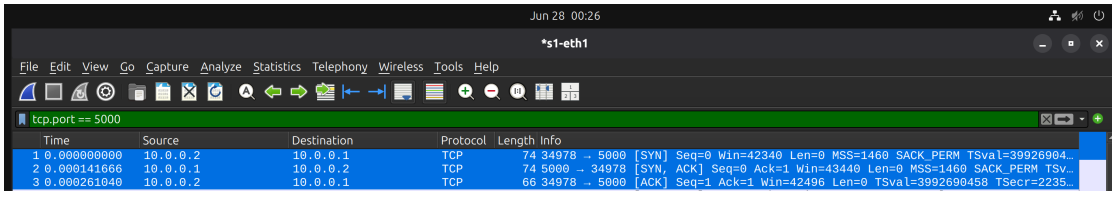


Figure 1: TCP three-way handshake: SYN, SYN-ACK, ACK

The handshake confirms successful connection establishment before data exchange begins.

5.2 Chat Message Packet

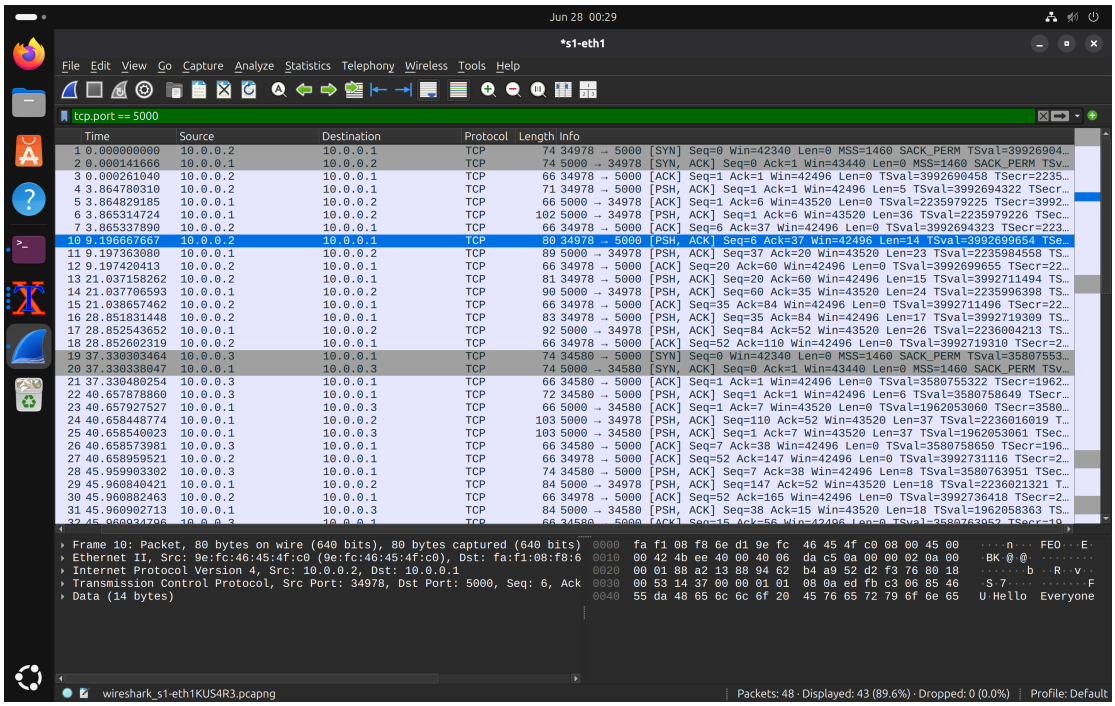


Figure 2: Chat message packet carrying application payload

This packet shows actual chat data being transmitted with TCP PSH and ACK flags.

5.3 Server Broadcast Packet

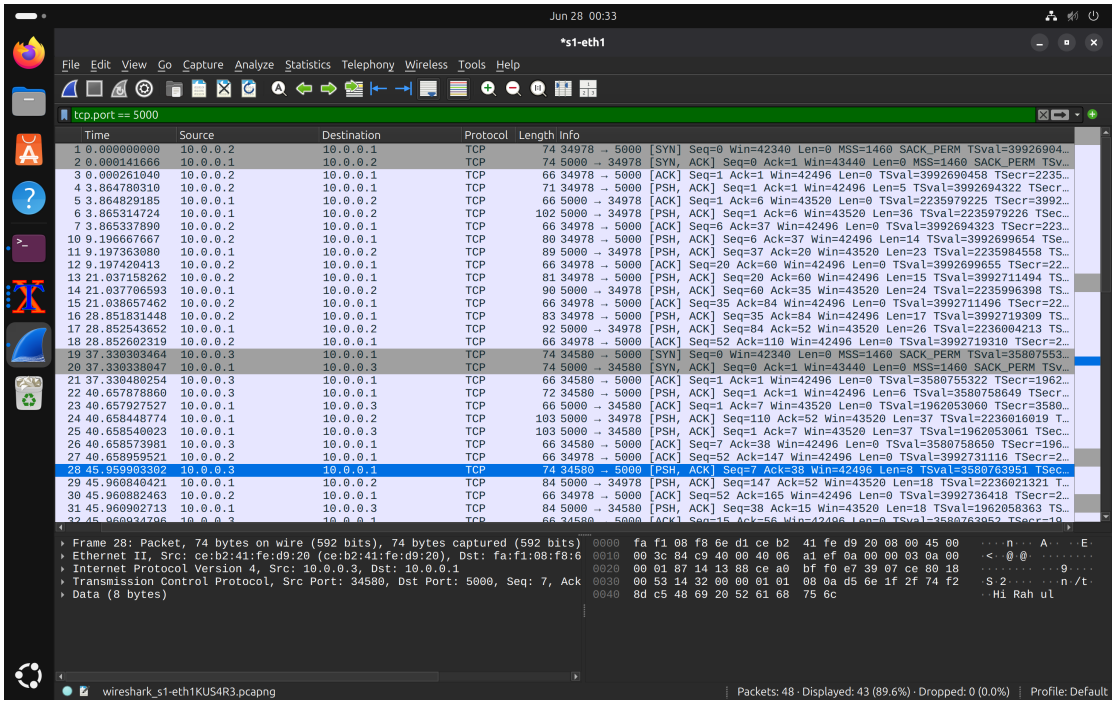


Figure 3: Server broadcasting the same message to multiple clients

The screenshot confirms that the server sends replicated message payloads to all connected clients.

5.4 TCP Connection Termination

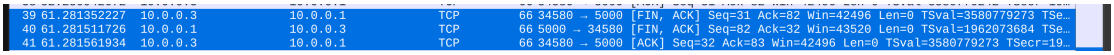
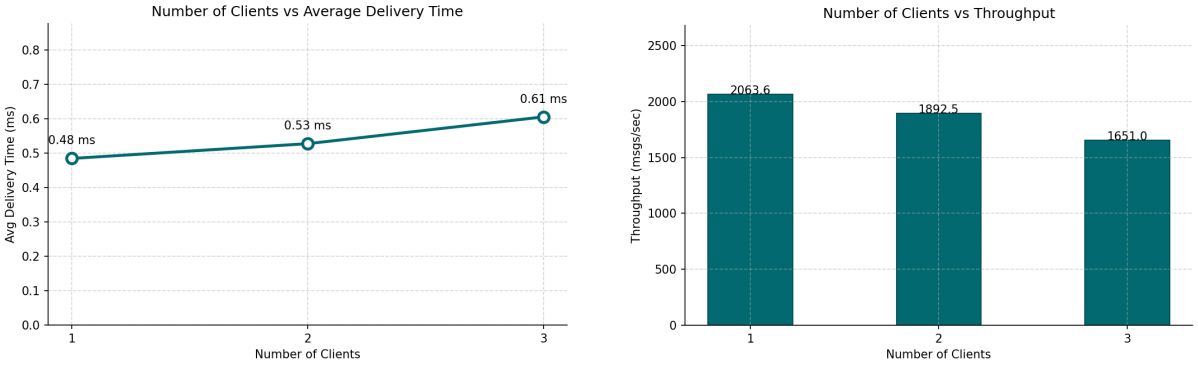


Figure 4: TCP connection termination using FIN/ACK exchange

The FIN/ACK sequence verifies graceful disconnection of the client session.

6 Graph Analysis



(a) Clients vs Average Delivery Time (b) Clients vs Throughput

Figure 5: Graphs generated from `performance_results.csv`

The first graph shows that average message delivery time rises from 0.485 ms to 0.606 ms as the client count increases from 1 to 3. The second graph shows that throughput decreases from 2063.558 msgs/sec to 1651.028 msgs/sec. These trends are expected because broadcasting to more clients increases the server's load.

7 Reflection Answers

Q1. Why is a multi-threaded server preferred over a single-threaded server?

A single-threaded server can handle only one blocking receive operation at a time. If one client is idle or slow, every other client must wait. A multi-threaded server assigns one thread to each client, allowing the operating system to schedule them independently and improving responsiveness.

Q2. What challenges arise when multiple clients communicate simultaneously?

The main challenge is shared-state synchronization. Multiple threads may try to add, remove, or iterate through the client dictionary at the same time, which can cause race conditions or runtime errors. This problem is solved using a mutex lock. Another challenge is handling abrupt disconnections gracefully.

Q3. How does TCP ensure reliable delivery of chat messages?

TCP uses sequence numbers, acknowledgments, retransmissions, checksums, and flow control. These mechanisms ensure that data arrives in order, without corruption, and is retransmitted if lost.

Q4. What information from Wireshark helped verify the application behavior?

Wireshark verified the SYN/SYN-ACK/ACK handshake, PSH/ACK packets carrying chat messages, repeated packets from the server during broadcast, and FIN/ACK packets during connection teardown. Together, these packet-level observations confirmed that the application behaved exactly as intended.

8 Conclusion

This assignment successfully implemented a multi-client chat server using Python, TCP sockets, and threads. The server accepted multiple clients simultaneously, registered usernames, broadcast messages, logged events, and measured performance under different client loads. The results showed low latency, high throughput, and correct TCP behavior verified through Wireshark captures.